Artefactos del Aprendizaje

Ese baúl de herramientas que todos usan, nadie entiende y aun así juran que sin él la trama no avanza

Daniel Barajas Higuera

MLEARN



Contenido

1. Repaso
2. scikit-learn
3. Ejemplos de uso

Artefactos que ya conocemos

No son nuevos: ya los hemos usado en prácticas anteriores. Pero ahora vamos a verlos bajo otra luz:

- No son simples librerías: son la base de casi todo el ecosistema de ML en Python.
- Cada uno cumple un rol esencial: cálculo, organización, visualización.
- Juntos parecen encajar en un diseño más grande...

Lo que antes eran simples herramientas, hoy revelan su verdadero poder.

NumPy — “El Ábaco Infinito”

Un dispositivo ancestral que hace cálculos más rápido de lo que cualquiera podría. Representa el poder del cálculo base.



Iniciado **2005** (evolución de Numeric y numarray; liderazgo de Travis Oliphant).

Cómo funciona

- Arreglo n–dimensional `ndarray` con **memoria contigua** y **operaciones vectorizadas** (BLAS/LAPACK).
- **Broadcasting** y tipos numéricos eficientes.

Para qué sirve

- Cálculo base: álgebra lineal, estadísticas elementales, generación de datos.
- Capa de bajo nivel para bibliotecas de alto nivel.

pandas — “El Espejo de Datos”

Un artefacto que transforma el caos en orden aparente: convierte montones de datos en espejismos de tablas limpias y legibles.



Iniciado **2008** (creado por Wes McKinney).

Cómo funciona

- Basado en **Series** y **DataFrames** construidos sobre NumPy.
- Ofrece índices, **alineación por etiquetas**, joins, **groupby**, operaciones con ventanas.

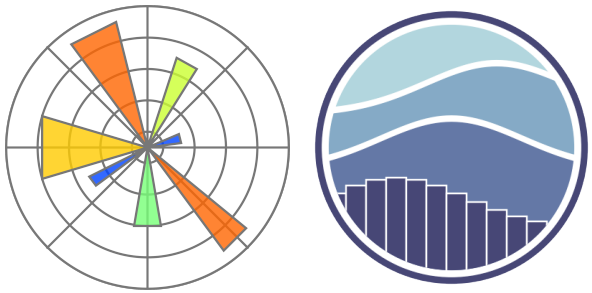
Para qué sirve

- Limpieza, transformación y preparación tabular.
- Entrada/salida a múltiples formatos; puente hacia algoritmos de ML.

matplotlib / seaborn — “Prismas de la Visualización”

Prismas que descomponen la oscuridad de los datos en formas visibles: desde gráficos sencillos hasta retratos complejos de la realidad.

matplotlib: creado en **2003** por John D. Hunter.
seaborn: lanzado en **2014** por Michael Waskom.



Cómo funciona

- **matplotlib**: API de bajo nivel, estilo *stateful* o *object-oriented*.
- **seaborn**: alto nivel sobre matplotlib, **integración con pandas**, estilos y plantillas por defecto.

Para qué sirve

- Exploración visual: histogramas, dispersión, box/violin, mapas de calor, pairplots.

Statsmodels — “El Reloj de la Inferencia”

Un artefacto que detiene el tiempo para examinar cada relación: expone causas, estima parámetros y contrasta hipótesis con precisión ritual.



Iniciado **2010** (dirigido por Skipper Seabold y Josef Perktold).

Cómo funciona

- Modelos lineales y generalizados (OLS, GLM, etc.).
- Métodos para **series de tiempo** y estimación econométrica.
- Pruebas de hipótesis, intervalos de confianza y diagnósticos.

Para qué sirve

- Análisis estadístico riguroso y explicativo.
- Aporta **inferencia** donde otros solo predicen.

El Camino Hacia la Esencia

Hemos repasado cuatro artefactos fundamentales:

- **NumPy** — el cálculo veloz que da forma a los cimientos.
- **pandas** — el espejo que ordena el caos en estructuras legibles.
- **matplotlib / seaborn** — los prismas que descomponen los datos en luz visible.
- **Statsmodels** — el mecanismo que revela huellas ocultas en el devenir de los datos.

Pero todavía falta una pieza. La esencia que da coherencia al conjunto...

Contenido

1 Repaso

2 scikit-learn

3 Ejemplos de uso

scikit-learn — “La Esencia del Aprendizaje”

Un artefacto que condensa la coherencia del aprendizaje clásico: unifica algoritmos, preprocesamiento y validación bajo un mismo lenguaje compartido.



Qué es

- Biblioteca de **machine learning clásico en Python**, iniciada en 2007.
- Estándar de facto para enseñanza, prototipado y producción ligera.

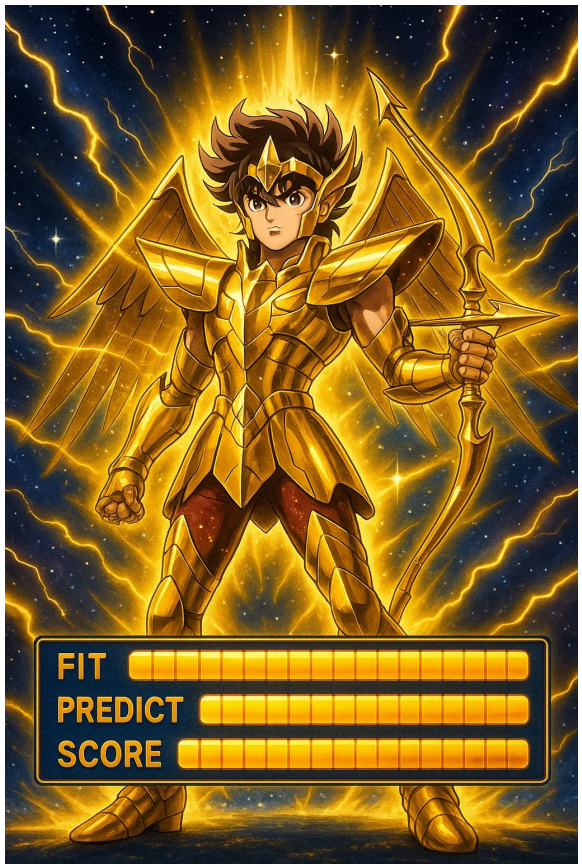
Para qué se utiliza

- Entrenamiento y evaluación de modelos supervisados y no supervisados.
- Preprocesamiento, selección de características y pipelines completos.
- Validación y comparación justa entre modelos.

Programación Orientada a Objetos natural

Cada modelo en scikit-learn es una **clase**:

- Se instancia como un caballero que recibe su armadura.
- Métodos que concentran el "cosmos":
  - `fit()` = entrenamiento, acumula poder.
  - `predict()` = ataque, libera la energía.
  - `score()` = evaluación, mide la fuerza lograda.
- La API es uniforme: todos los caballeros siguen el mismo juramento.



Abstracción en scikit-learn

- Los estimadores comparten una interfaz estándar: `fit`, `predict`, `transform`.
- El usuario no necesita saber los detalles internos de cada algoritmo.
- Esto permite intercambiar modelos sin cambiar el flujo de trabajo.
- Ejemplo: lo mismo da un `LogisticRegression` que un `RandomForest`, ambos saben entrenar y predecir.

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X = [[0,1],[1,0],[0,-1],[-1,0]]
y = [0,1,0,1]

scaler = StandardScaler()
clf = LogisticRegression()

# Ambos siguen la interfaz estándar
scaler.fit(X)
clf.fit(scaler.transform(X), y)
clf.predict([[0.5,0.5]])
```

Encapsulamiento en scikit-learn

- El encapsulamiento es débil: no hay modificadores de acceso como en Java.
- Se usa convención: atributos internos terminan en _ o __.
- Los usuarios consultan resultados en atributos públicos como `coef_` o `intercept_`.
- La regla no escrita: si lleva guion bajo, no lo toque directamente.

```
from sklearn.linear_model import LogisticRegression

X = [[0,1],[1,0],[0,-1],[-1,0]]
y = [0,1,0,1]

m = LogisticRegression().fit(X, y)

# Estos atributos están "expuestos" pero con convención
print(m.coef_)
print(m.intercept_)

# Internamente hay otros con doble guion bajo, no pensados para tocar.
```

Herencia en scikit-learn

- Presente a nivel de librería, no tan visible para el usuario.
- Todos los estimadores heredan de `BaseEstimator`.
- Los clasificadores heredan de `ClassifierMixin`, los regresores de `RegressorMixin`.
- Esto asegura que compartan la misma API, aunque como usuario raramente extienda clases.

```
from sklearn.linear_model import LogisticRegression
from sklearn.base import BaseEstimator, ClassifierMixin

m = LogisticRegression()

print(isinstance(m, BaseEstimator))     # True
print(isinstance(m, ClassifierMixin))   # True
```

Polimorfismo en scikit-learn

- Distintos modelos responden igual a `fit`/`predict`.
- Permite probar varios algoritmos sin cambiar el código.
- Es la base para bucles automáticos de benchmarking.
- Ejemplo: Logistic Regression y Random Forest ejecutan la misma secuencia de llamadas.

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

X = [[0,1],[1,0],[0,-1],[-1,0]]
y = [0,1,0,1]

models = [LogisticRegression(), RandomForestClassifier()]
for model in models:
    model.fit(X, y)
    print(model.predict([[0.5,0.5]]))
```

Composición en scikit-learn

- Los objetos se forman de piezas: `Pipeline`, `FeatureUnion`, `ColumnTransformer`.
- Cada etapa es un estimador independiente.
- La composición reduce código repetido y asegura consistencia en el flujo.
- Ejemplo: un pipeline con escalador y clasificador.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("sc", StandardScaler()),
    ("clf", LogisticRegression())
])

X = [[0,1],[1,0],[0,-1],[-1,0]]
y = [0,1,0,1]

pipe.fit(X, y)
print(pipe.predict([[0.5,0.5]]))
```

Delegación en scikit-learn

- Un objeto expone la interfaz pero no hace el trabajo: lo encarga a otro.
- Ejemplo: `GridSearchCV` mantiene `fit`/`predict`, pero las llamadas se delegan al mejor estimador.
- También ocurre en `OneVsRestClassifier`, que delega en varios clasificadores internos.
- Delegación = composición + transferencia de responsabilidad.

```
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)

grid = GridSearchCV(SVC(), {"C": [0.1, 1, 10]}, cv=3)
grid.fit(X, y)

print(grid.best_estimator_)
print(grid.predict(X[:3]))
```

POO en scikit-learn

Pilar	**Ejemplo**	**Nivel de uso**
Abstracción	Interfaz común `fit`/`predict`/`transform`.	🔥🔥🔥
Encapsulamiento	Atributos internos con guion bajo (`coef_`, `intercept_`).	🔥
Herencia	Herencia de `BaseEstimator`, `ClassifierMixin`, etc.	🔥🔥
Polimorfismo	Diferentes algoritmos responden igual a `fit`/`predict`.	🔥🔥🔥
Composición	`Pipeline`, `FeatureUnion`, `ColumnTransformer`.	🔥🔥🔥🔥🔥
Delegación	`GridSearchCV`, `OneVsRestClassifier`.	🔥🔥🔥🔥🔥

Escala de Cosmos: 1 = 🔥 5 = 🔥🔥🔥🔥🔥

Contenido

1 Repaso

2 scikit-learn

3 Ejemplos de uso

Ejemplo: Regresión Lineal en scikit-learn

```python
from sklearn.linear_model import LinearRegression
import numpy as np

# Datos simples
X = np.array([[40],[25],[35],[60],[55]])
y = np.array([280,150,220,380,420])

# Crear el "caballero"
modelo = LinearRegression()

# Entrenar su cosmos
modelo.fit(X, y)

# Usar su ataque
print(modelo.predict([[43]]))
```



Ejemplo: KNN en scikit-learn

```python
from sklearn.neighbors import KNeighborsClassifier

# Datos simples
X = [[0,0],[1,1],[2,2],[3,3]]
y = [0,0,1,1]

# Crear el caballero
knn = KNeighborsClassifier(n_neighbors=3)

# Entrenar
knn.fit(X, y)

# Predecir clase
print(knn.predict([[1.5, 1.5]]))
```



Ejemplo: SGDRegressor en scikit-learn

```python
from sklearn.linear_model import SGDRegressor
import numpy as np

# Datos simples (relación lineal)
X = np.array([[40],[25],[35],[60],[55]])
y = np.array([280,150,220,380,420])

# Crear el caballero
sgd_reg = SGDRegressor(max_iter=1000, tol=1e-3)

# Entrenar
sgd_reg.fit(X, y)

# Predecir valor
print(sgd_reg.predict([[43]]))
```



Pipelines y Validación

Pipelines

- Encadenar pasos: escalado $\rightarrow$ modelo.
- Evita fugas de datos, garantiza reproducibilidad.

Validación

- `train_test_split`, `cross_val_score`.
- `GridSearchCV` y `RandomizedSearchCV`.



¿Hay algo más detrás?

- A simple vista, parece “solo una librería”.
- Pero conecta algoritmos, preprocesamiento, métricas y validación.
- Es la esencia invisible que enlaza a los demás artefactos, como el **cosmos** que da fuerza y sentido en las batallas.

Su verdadero lugar en el universo aún está por revelarse.

EL VERDADERO PODER NO HA SIDO REVELADO…



**PERO SI NO PROGRAMAS POR TU CUENTA,
SERÁS ARRASTRADO AL
INFRAMUNDO DE LA REPETICIÓN**